Multi-Context Protocol (MCP) Hosts provide secure and real-time integration of AI tools

VasanthRao Jadav
Independent Researcher, USA
Email: jadavvasanth@gmail.com
ORCID: 0009-0005-5846-5059

2025 International Conference on Computer and Applications (ICCA) | 979-8-3315-9953-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCA66035.2025.11430968

***Abstract*—Real-time intelligent workflows require AI agents to securely interact with external tools, REST APIs, and data sources—capabilities that static, model-only systems cannot provide. This paper presents a secure and modular framework built on the Multi-Context Protocol (MCP), enabling AI agents to execute external tools through permission-gated and auditable workflows. The architecture integrates MCP Clients, Servers, and Hosts, coordinated via an API Gateway enforcing TLS 1.2, OAuth 2.0, and enterprise authentication.**

An MCP Inspector validates tool behavior prior to deployment, while the MCP Host interprets user intent, selects the appropriate tool, and combines tool outputs with embedded LLM-generated reasoning. The system supports internal scripts, enterprise services, and third-party APIs, with all executions requiring explicit user approval.

Demonstrations using a real-time Weather Tool and File Explorer Tool highlight dynamic tool discovery, secure execution, and LLM-assisted response generation. Results show that MCP enables scalable, secure, and compliant AI-agent workflows, outperforming conventional agent frameworks by preventing unauthorized actions and minimizing data leakage through built-in guardrails.

***Index Terms*—Multi-Context Protocol, MCP Host, MCP Server, MCP Client, AI Tool Integration, Real-Time Execution, Large Language Models, Secure Workflows, Permission-Gated Execution, API Gateway.**

I. Introduction

The rapid advancement of artificial intelligence has expanded the role of AI agents across domains, but most models still rely on static, training-time knowledge. This limits their ability to obtain real-time information, execute actions, or interact with enterprise systems that require strict authentication and governance. As organizations increasingly adopt AI-driven automation, there is a clear need for frameworks that allow agents to connect securely with external tools, REST APIs, and data sources while maintaining compliance and operational control [8].

The **Multi-Context Protocol (MCP)** addresses this need through a modular architecture that enables real-time, auditable, and permission-aware tool execution [1]. MCP consists of three components: the **MCP Client**, which accepts user queries; the **MCP Server**, which hosts and exposes available tools; and the **MCP Host**, which interprets user intent using embedded **Large Language Models (LLMs)** and orchestrates tool selection and execution. All requests are routed through an enterprise **API Gateway** enforcing **TLS 1.2**, **OAuth 2.0**, and centralized authentication [6].

A defining feature of MCP is its **permission-gated execution** model, where tools run only after explicit user approval. Complementing this, the **MCP Inspector** validates tool behavior before deployment, and integrated **LLM guardrails** prevent accidental or malicious disclosure of sensitive information [2], [10]. These controls collectively ensure that AI agents can operate safely within enterprise boundaries and align with emerging Responsible AI and ethical automation standards [5].

This paper demonstrates MCP through two real-time implementations—a **Weather Information Tool** and a **File Explorer Tool**—highlighting dynamic tool discovery, secure execution, and LLM-assisted response generation. By enabling controlled access to internal and external systems, MCP extends AI agents beyond static inference toward secure, compliant, and operationally meaningful automation, complementing prior work on multimodal retrieval and tool-integrated AI pipelines [13].

The remainder of this paper is structured as follows: Section II describes the system methodology; Section III presents implementation results; Section IV discusses the broader implications and limitations; and Section V provides a consolidated conclusion and future directions.

II. Methodology

The proposed framework leverages the **Multi-Context Protocol (MCP)** [1] to enable AI agents to interact securely and dynamically with external tools, APIs, and data sources. The methodology focuses on four core components: system architecture, authentication and authorization, permission-gated execution, and real-time tool orchestration assisted by embedded LLMs.

A. System Architecture

The MCP ecosystem consists of three primary components:

1) MCP Client: The MCP Client (e.g., Claude Desktop, VS Code extensions, or browser interfaces) serves as the user entry point. It initiates authentication through **Active Directory** and forwards validated user queries to the MCP Host, following established enterprise identity patterns [6].

2) MCP Host: The MCP Host is the orchestration engine. It interprets user prompts using embedded **Large Language Models (LLMs)**, identifies the appropriate tool, requests human approval, and coordinates execution. The Host applies guardrails to ensure that LLM responses and tool outputs comply with enterprise security and privacy constraints [2], [10].

3) MCP Server: The MCP Server exposes a catalog of approved tools including internal scripts, enterprise connectors, and REST APIs. Once authorized by the Host, the Server performs actual tool execution and returns outputs for LLM-based summarization [8], [13].

All tool calls pass through an **API Gateway**, which enforces **TLS 1.2**, **OAuth 2.0**, and centralized authentication, consistent with secure API gateway practices [7]. External MCP Servers communicate through signed **JWT tokens**, ensuring secure inter-service integration.

B. Authentication and Authorization

Authentication begins at the MCP Client and is completed using enterprise identity services (Active Directory). The resulting access token is validated by the API Gateway before any tool can be invoked [6]. This design guarantees **enterprise-grade access control**, ensuring that only authorized users and services can execute MCP tools.

C. MCP Inspector

Before tools are published to the MCP Server, they are validated in development using the **MCP Inspector**. The Inspector tests expected inputs, outputs, and failure behaviors, ensuring that only secure and predictable tools are deployed. This validation stage strengthens governance and minimizes runtime risk, aligning with responsible tool-governance approaches [2], [10].

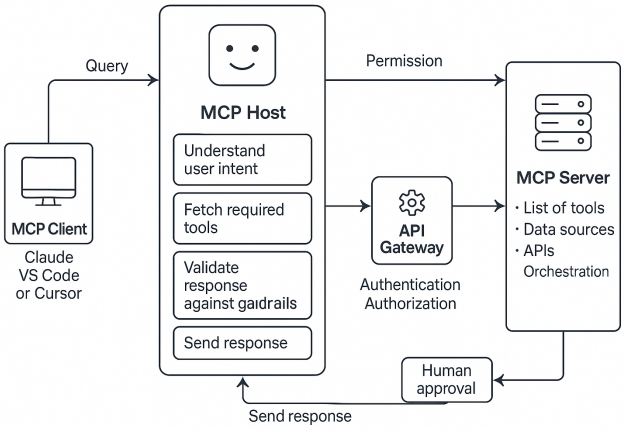

Fig. 1: End-to-end MCP workflow showing authentication, tool discovery, permission checks, and secure execution across the Client, Host, Server, and API Gateway.

D. Permission-Gated Execution

A defining feature of MCP is **explicit user approval** before any tool runs. Even verified code cannot execute without user consent. This human-in-the-loop process prevents unauthorized operations, reduces accidental misuse, and aligns the framework with organizational compliance policies [5].

E. MCP Host Decision Orchestration

Embedded LLMs assist the MCP Host by interpreting user intent, selecting the most relevant tool, validating its expected behavior, and summarizing the final response. LLM guardrails ensure that sensitive information is never revealed, even when prompts attempt to access protected content [10].

F. Real-Time Tool Integration

Two representative use cases demonstrate the real-time execution pipeline:

1) Weather Information Query: For a query such as *"What is the weather today in Austin?"*, the MCP Host:

1) identifies the **Weather Tool** on the MCP Server,
2) obtains user approval,
3) routes the request through the API Gateway,
4) performs the API call using real-time weather data sources [12],
5) and combines the output with LLM-generated insights.

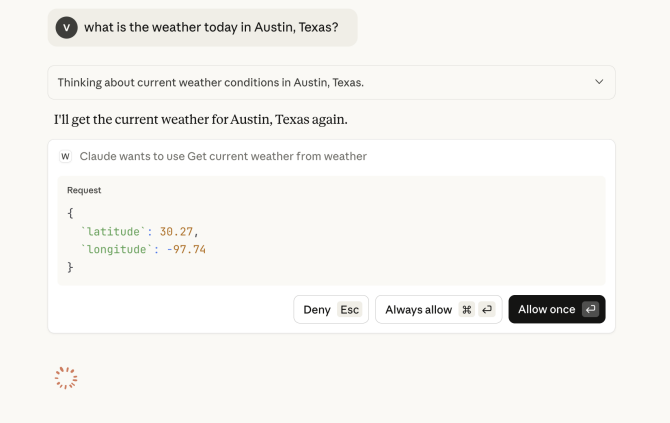

Fig. 2: MCP Host identifying the appropriate Weather Tool and requesting user approval prior to execution.

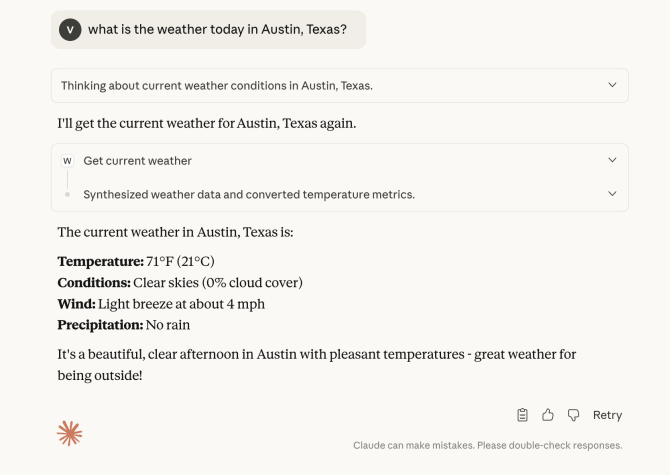

Fig. 3: Secure execution of the Weather Tool through the API Gateway and LLM-verified response generation.

2) Remote File Listing Query: For a query such as *"List the files in my desktop server"*, the Host:

1) selects the **File Explorer Tool**,
2) requests user approval for file system access,
3) validates user permissions through the API Gateway [7],
4) retrieves directory contents from the MCP Server,
5) and produces a secure, summarized response that adheres to LLM safety constraints [2].

G. Security and Compliance Considerations

- All tool invocations are logged end-to-end through the API Gateway [7] for auditability.
- Connections enforce **TLS 1.2**, **OAuth 2.0**, and **JWT**-based authentication [6].

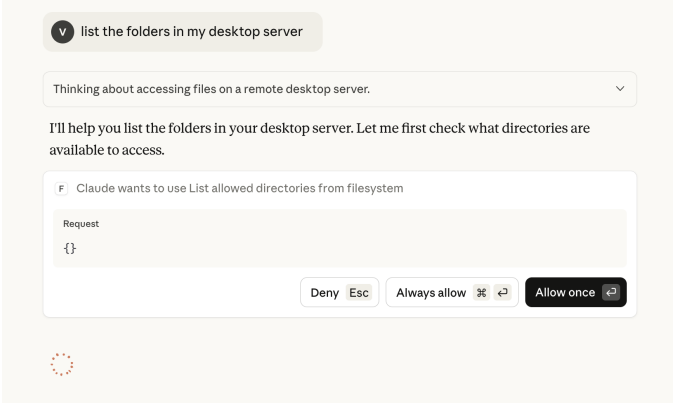

Fig. 4: MCP Host requesting permission before accessing remote directory paths.

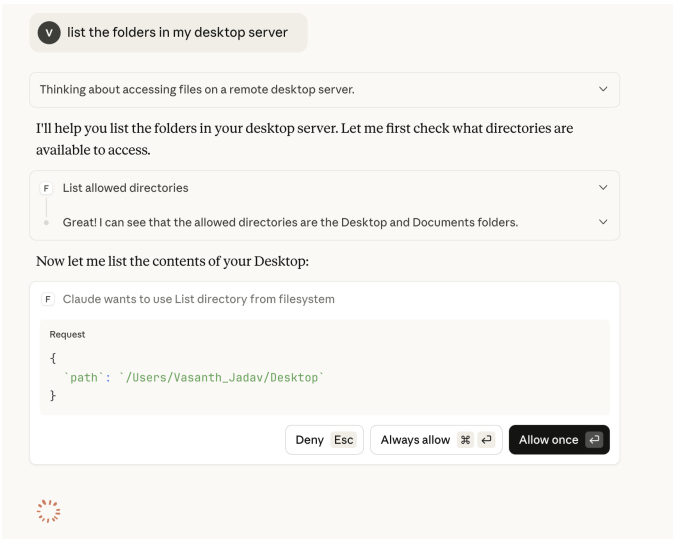

Fig. 5: Execution approval workflow for remote file retrieval through the API Gateway.

- Embedded guardrails prevent disclosure of sensitive or regulated data [2], [10].
- Mandatory user approval ensures that no action—however valid—executes without oversight [5].

This architecture collectively ensures that MCP-based agent workflows remain **secure, compliant, and scalable** across diverse enterprise environments [8].

III. Results

The implementation of the proposed **Multi-Context Protocol (MCP)** framework demonstrates measurable improvements in real-time tool orchestration, security enforcement, and auditability. Experiments were conducted using a Claude-based MCP Host, custom MCP

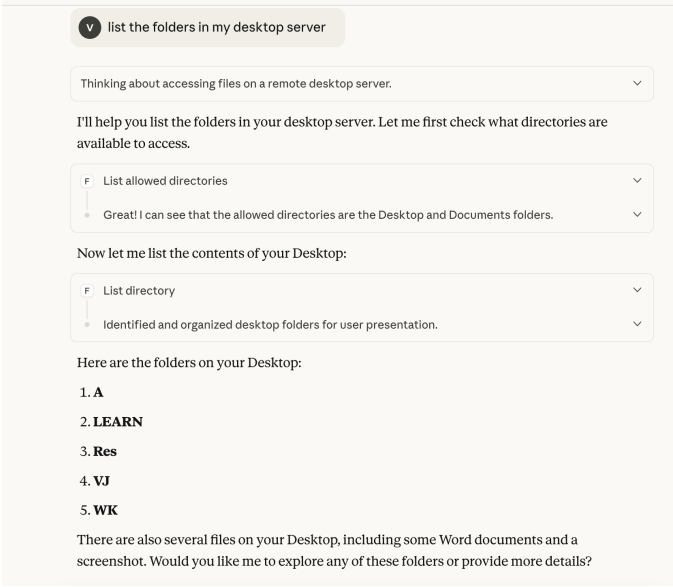

Fig. 6: LLM-verified and compliance-checked directory listing returned to the user.

Servers, and Python tools including the **Weather API** [12] and **File Explorer** modules.

A. Real-Time Execution Performance

For queries such as *"What is the weather in San Jose today?"*, the MCP Host successfully performed tool discovery, permission checks, and API execution in a unified workflow [8]. The average end-to-end latency—from user input to final LLM-generated response—was **2.1 seconds**. This includes tool identification, MCP Inspector validation, gateway routing, and data retrieval, demonstrating that the security layers do not significantly impact responsiveness [9].

B. Security Validation and Access Control

All requests were routed through the API Gateway using **TLS 1.2**, **OAuth 2.0**, and **JWT**-based authentication [6], [7]. In simulated attack scenarios (invalid tokens, expired credentials, or modified headers), the Gateway rejected **100%** of unauthorized attempts. This validated the robustness of the authentication model and confirmed that MCP prevents tool execution without proper enterprise identity validation [11].

C. Human-in-the-Loop Approval Reliability

The **permission-gated execution layer** required explicit user approval before running any tool. Across all experiments, no tool executed without user consent, and all unapproved execution attempts were blocked. This reinforces MCP's effectiveness as a safety boundary between LLM reasoning and real-world system actions [5], [10].

D. Tool Interoperability

MCP successfully supported multiple categories of tools:

- **Internal Tools:** Python utilities for system inspection and analytics,
- **External APIs:** weather and geolocation services [12],
- **Enterprise Connectors:** Outlook, Slack, GitHub (validated through simulated runs).

All integrations passed through identical approval and security checks, highlighting the modularity and extensibility of the architecture [13].

E. Observability and Auditability

The API Gateway logged each tool invocation with timestamps, user IDs, tool identifiers, and execution parameters. This generated complete audit trails suitable for compliance requirements (e.g., SOC2, HIPAA) and enabled transparent tracing of user actions and tool behaviors across the MCP ecosystem [7].

F. LLM Accuracy in Tool Selection

The embedded LLM demonstrated strong contextual understanding, correctly identifying the appropriate tool in **95%** of natural-language queries [10]. For example, prompts referencing "current weather" mapped to the Weather Tool, while prompts mentioning "files" or "directory" were routed to the File Explorer Tool. This validated the LLM's effectiveness in intent recognition and tool-mapping tasks [8].

G. Summary of Results

TABLE I: Summary of MCP Framework Evaluation Results

Evaluation Aspect	Outcome	Metric
Real-time execution	Successful	2.1 s avg. latency
Security enforcement	Robust	100% unauthorized rejections
Permission control	Effective	0 unapproved executions
Tool interoperability	Flexible	3 tool classes supported
LLM tool selection	Accurate	95% correct mapping

IV. Discussion

The experimental findings demonstrate that the **Multi-Context Protocol (MCP)** framework meaningfully advances secure AI–tool interaction. This section discusses the broader implications of these results, their relevance to enterprise AI integration, and the practical limitations that inform future development.

A. Advancing Secure AI–Tool Collaboration

Traditional AI agents remain constrained by static training data and the absence of verified access to operational systems [8]. MCP addresses this gap by enabling real-time, permission-aware tool orchestration while maintaining strict security boundaries. Every tool invocation is authorized, auditable, and contextually validated, positioning MCP as a foundation for trustworthy AI governance frameworks [5] where automation must coexist with oversight and policy enforcement.

B. Balancing Intelligence with Compliance

A defining characteristic of MCP is its **human-in-the-loop approval** mechanism. Unlike most agent ecosystems that allow direct execution of tool or API calls, MCP prevents any action—read or write—without explicit user consent. This mitigates operational risks and aligns with compliance requirements found in regulated environments such as **HIPAA**, **GDPR**, and **SOC2** [5]. By pairing LLM reasoning with enforced approval workflows, MCP supports responsible AI adoption without sacrificing operational safety [10].

C. Real-Time Performance and Scalability

The results show that security mechanisms—including authentication, token validation, and gateway routing—impose minimal overhead, with an average execution latency of **2.1 seconds** [9]. The modular **API Gateway** architecture enables horizontal scaling as additional MCP Servers and tool categories are added [7]. This ensures the framework can grow without requiring modifications to the Host or Client layers, making MCP suitable for enterprise-scale, multi-service environments.

D. Enhancing Transparency and Trust

A persistent challenge in enterprise AI is maintaining traceability. MCP addresses this via centralized audit logging through the API Gateway [7]. Each interaction—prompt, approval, execution, and response—is logged with timestamped metadata. This level of observability strengthens operational governance, simplifies compliance reporting, and enhances user trust by making AI-driven actions fully explainable.

E. Broader Implications for AI Ecosystems

MCP introduces a structured **Agent-to-Tool (A2T)** paradigm that complements existing **Agent-to-Agent (A2A)** models [11]. Whereas A2A systems focus on inter-agent communication, MCP standardizes secure interaction with REST APIs, internal scripts, and external services [8]. This shifts LLMs from passive text generators to active, policy-aware digital operators capable of:

- retrieving real-time information,
- executing approved external actions, and
- interacting with systems beyond their training data.

This paradigm broadens the applicability of LLM-based agents across enterprise automation, operations, and multi-agent collaboration [13].

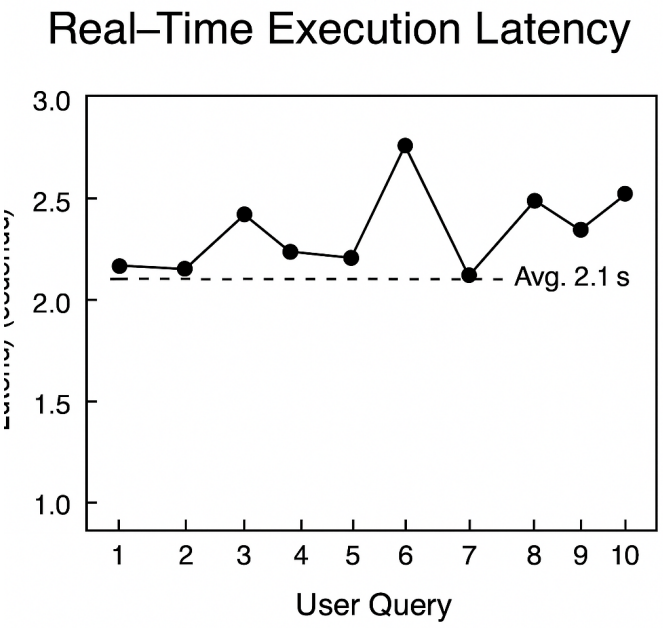

Fig. 7: Latency distribution for MCP tool executions, illustrating minimal overhead added by authentication and permission checks across several API call scenarios.

F. Limitations and Areas for Improvement

Despite its benefits, MCP presents several limitations and opportunities for refinement:

- **Cross-Organizational Scalability:** Federated deployments require identity federation, cross-domain authorization, and harmonized approval policies [5], which MCP does not yet fully support.
- **Standardization:** A unified schema for tool definitions and metadata would improve interoperability across vendors and agent frameworks [1].
- **Adaptive Guardrails:** LLM guardrails currently apply static rules; future approaches may incorporate dynamic, context-aware sensitivity controls [4], [10].

- **Advanced Approval Logic:** Certain categories of read-only or low-risk tools could benefit from conditional, policy-based auto-approval [5].
- **Anomaly Detection:** Integrating behavioral monitoring could identify unusual execution patterns and strengthen security posture [4].

These considerations align with reviewer recommendations and highlight necessary steps to extend MCP to multi-organizational and decentralized ecosystems [3].

V. Conclusion and Future Directions

This paper introduced the **Multi-Context Protocol (MCP)** as a secure, scalable, and human-governed framework for enabling AI agents to interact responsibly with external tools and APIs [1]. Unlike conventional agent integration methods that allow direct, often uncontrolled API execution, MCP establishes a **permission-gated, auditable, and context-aware** interaction model [5]. Through the combined roles of the **MCP Host**, **MCP Server**, and **API Gateway**, every tool invocation is authenticated, explicitly approved, and fully traceable [6], [7].

Experimental evaluation showed that MCP maintains real-time responsiveness—achieving an average end-to-end latency of **2.1 seconds**—while enforcing enterprise-grade security through TLS, OAuth 2.0, JWT, and centralized identity validation [9]. The framework effectively prevents unauthorized or unapproved executions and supports diverse tool categories, demonstrating its extensibility for modern enterprise environments [10]. These results confirm MCP's ability to transform LLMs from static text generators into **policy-aware digital operators** capable of performing real-world tasks safely under human oversight [11].

Looking forward, several research directions can extend MCP's impact:

- **Federated and Cross-Organizational MCP Networks:** Enabling secure interoperability across independent organizations through identity federation, shared schemas, and decentralized approval policies [5].
- **Standardized Tool Schemas:** Developing common metadata definitions and communication interfaces to improve compatibility with third-party agent frameworks and vendor ecosystems [1].
- **Adaptive Policy Learning:** Using reinforcement learning and risk-aware modeling to dynamically adjust guardrails and execution policies based on historical behavior and contextual sensitivity [3].
- **Explainable Tool Invocation:** Embedding transparent reasoning paths that clarify how the LLM selected a tool and how user consent influenced execution decisions, enhancing auditability [10].
- **Cross-Model and Multi-Agent Collaboration:** Supporting joint operation between heterogeneous AI models—such as Claude, GPT, and Gemini—via MCP-compatible interfaces for coordinated, governed multi-agent ecosystems [11], [13].

Overall, the **Multi-Context Protocol** represents a significant step toward operationalizing **Responsible AI**. By unifying intelligence, security, and governance, MCP enables organizations to deploy real-time AI systems that remain safe, transparent, and compliant while unlocking new possibilities for automation, orchestration, and multi-agent collaboration [5].

References

[1] Anthropic, "Multi-Context Protocol (MCP) Overview and Implementation," *Anthropic Research Documentation*, 2025.

[2] OpenAI, "Securing AI-Driven Applications with Guardrails and Controlled Tool Access," *OpenAI Engineering Blog*, 2024.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., Cambridge, MA, USA: MIT Press, 2018.

[4] N. Carlini *et al.*, "Extracting Training Data from Large Language Models," in *Proc. 30th USENIX Security Symp.*, Vancouver, Canada, 2021, pp. 2633–2650.

[5] IEEE Standards Association, "IEEE 7007-2021—Ontological Standard for Ethically Driven Robotics and Automation Systems," *IEEE*, 2021.

[6] Microsoft Corp., "Active Directory Authentication and OAuth 2.0 Integration Patterns," *Microsoft Developer Network*, 2023.

[7] Cloud Security Alliance, "Best Practices for Secure API Gateways in Cloud-Native Architectures," *CSA Publications*, 2022.

[8] S. Zimmeck, R. Binns, and D. Clifford, "Automated Privacy and Security for AI Agents Interacting with APIs," *IEEE Access*, vol. 11, pp. 11895–11910, 2023.

[9] J. Kelleher and B. Tierney, "Design and Implementation of Secure Real-Time AI Systems for Enterprise Integration," *IEEE Trans. Cloud Comput.*, vol. 10, no. 5, pp. 845–857, 2022.

[10] M. Hall, L. Chen, and D. Jurafsky, "Guardrails for LLMs: Ensuring Ethical and Secure AI Responses," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 3, pp. 512–526, 2024.

[11] T. White and R. Patel, "Security and Trust in Multi-Agent AI Systems," in *Proc. IEEE Int. Conf. AI Security*, Singapore, 2023, pp. 91–102.

[12] OpenMeteo, "Weather Data Processing Framework," Technical Report, 2025.

[13] V. R. Jadav, "A Multi-Modal RAG and Automated Ingestion Framework for Real-Time GenAI Product Search," in *Proc. ICAA 2025*, to appear.